4

Geometry of Binary Threshold Neurons and Their Networks

God eternally geometrizes.

Plato

4.1 Pattern Recognition and Data Classification

Pattern recognition is the "field concerned with machine recognition of regularities in noisy or complex environments [136]." Alternatively, pattern recognition is the "search for structure in data [33]." Ultimately, it involves placing an input pattern into one of possibly many decision classes.

The ability to recognize patterns effortlessly and efficiently, and take decisions based on the results of that recognition process is one of the distinguishing features of human intelligence. Recognition itself involves the ability to classify data into (known) categories. The fact that we can recognize, has to do with the fact that we can classify. Human beings have an innate ability to recognize complex patterns with effortless ease, based on categories created internally without conscious intervention. Examples abound: from the ability of a parent to recognize the cry of a child; to our being able to identify an individual from a silhouette, or a fast moving car at dusk. Consequently, real world applications are drawing increasingly upon the brain metaphor in the design of embedded pattern recognition systems that have the ability to efficiently and accurately sort and classify data. Applications such as these include ZIP code readers, DNA fingerprinting, medical diagnosis, occluded target recognition, signature verification, amongst a host of others.

Consider an application that is more familiar to us—weather forecasting. Assume that we have amassed a huge database of samples or *patterns*

derived from reliable measurements of various weather related parameters called *features*. A pattern is simply a feature vector. We presume that the available patterns are tagged and can be sorted into two categories: one corresponding to patterns that indicate rain; and the other corresponding to samples that do not. These categories are called *pattern classes*. A pattern class is a set of patterns that originate from the same source or real world process or probability distribution, and which share similar properties.

Patterns belonging to the same class share similar properties.

Suppose we want to design a *pattern classifier* that can *predict* whether or not it would rain on a particular day based on certain feature measurements taken over the previous twenty-four hours. The design of such a system necessarily involves the demarcation of a boundary that separates the two classes "rain indicated" and "rain not indicated" in question. Once such a *class separating boundary* or *decision boundary* is demarcated, prediction merely involves computing which side of the boundary a specific feature vector lies. A decision boundary could be as simple as a single straight line; or could have a more complicated piece-wise linear or non-linear functional form. Class boundaries partition the input pattern space into one or more *decision regions*. In the present example where there are only two decision regions in the pattern space, we say that the available set of data samples need to be *dichotomized*. In general, the classification problem may involve the separation of data into more than two classes.

Often, the patterns derived from real world measurements will *cluster* together in certain regions of the pattern space, and it may be easy to provide a demarcating boundary when the clusters are disjoint or non-overlapping. However, we must remember that the available data is only representative in nature—it is a sample of some larger, possibly unknown, distribution. Many data points that might exist in the real world are not included in this small sample set. Therefore, any demarcating class boundary must appropriately account for unseen data. Otherwise we cannot expect the performance of the classifier to be satisfactory when deployed in the real world. This complicates the problem considerably, and motivates the following statement of the basic pattern classification problem:

> *Given some samples of data measurements of a real world system along with correct classifications for patterns in that data set, make accurate decisions for future unseen examples.*

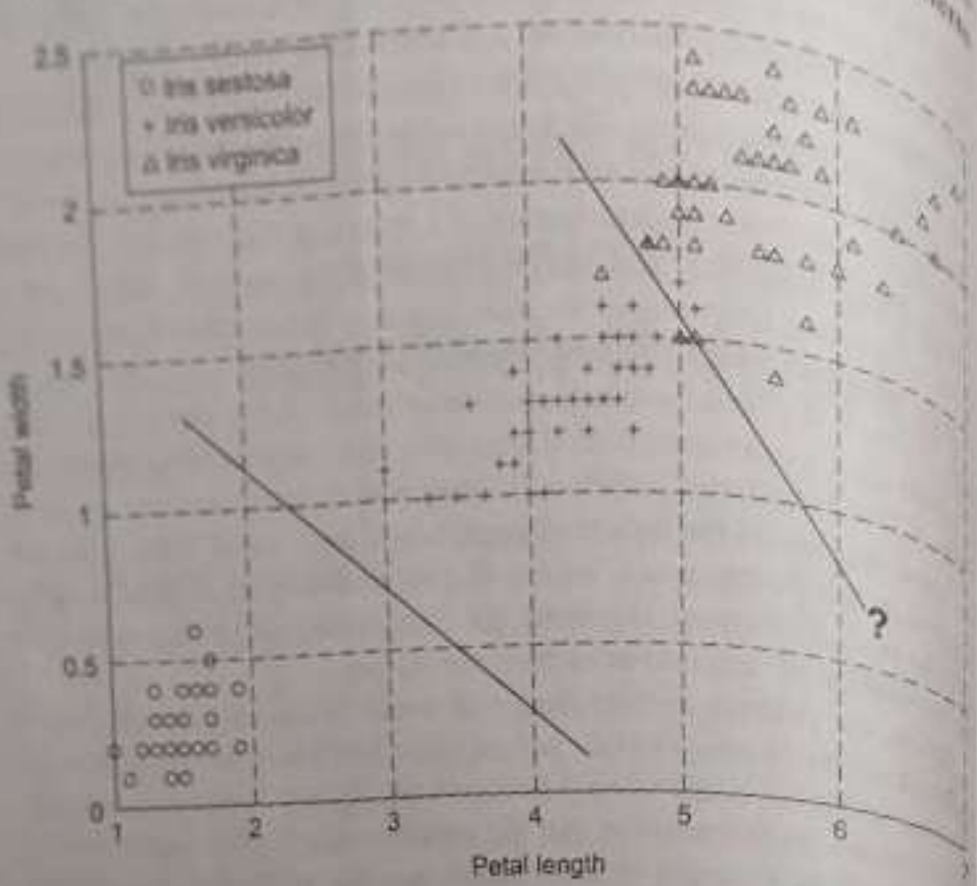
We continue to strengthen our ideas of the classification problem with a widely cited benchmark data set from the research literature.

4.1.1 Iris Data Classification Problem

The well known Iris data set comprises feature measurements of the iris flowers measured by Anderson [12] and popularized by Fisher [147]. It

The data is available by anonymous FTP from ftp.ics.uci.edu under the directory pub/machine-learning-databases. It is a four dimensional data and in Fig. 4.1 we view the projection of this data in the petal length–petal width subspace (one of the six possible sub-spaces).

consists of 150 patterns, each of four feature measurements: the sepal length ($x_{i,1}$), sepal width ($x_{i,2}$), petal length ($x_{i,3}$), and petal width ($x_{i,4}$). The data set is divided into three subsets of 50 patterns each for three different species of the flower: *iris sestosa*, *iris versicolor*, and *iris virginica*. These patterns are plotted on the $x_{i,3}$–$x_{i,4}$ subspace in Fig. 4.1.



Note: Notice how *iris sestosa* is separable in this projection by a single straight line from the *iris versicolor* and *iris virginica*, whereas *iris versicolor* and *iris virginica* are not.

Fig. 4.1 A view of the 4-dimensional Iris data projected on to the $x_{i,3}$–$x_{i,4}$ axes

Figure 4.1 shows us that it is easy to separate *iris sestosa* data from *iris versicolor* and *iris virginica* (in this subspace). You can easily see that it is much more difficult to decide where to place a separating line between *iris versicolor* and *iris virginica*. That is why the line has a question mark. Any placement of the straight line will cause some pattern to get misclassified. Of course, one can solve the problem with non-linear boundaries, and indeed the Iris data problem has been solved using non-linear feedforward neural network classifiers which will be the subject of discussion in Chapter 6. For the time being, do not forget that we are looking at only one projection of iris data. Class separation is facilitated by additional dimensions!

The assignment of labels to clusters is essentially a classification issue.

What can we conclude? Providing a clean cut classification of a data sample into a decision class is not so straightforward. For data samples that are well separated there is no problem. But when data samples mel

with one another, the delineation of class boundaries becomes a difficult problem to solve. Further, note that it is one issue to design a system to classify a given data set correctly, and another to accurately classify future unseen samples based on that design. This is further elaborated in Chapters 6 and 8.

4.1.2 Two-Class Data Classification Problem

We return to the question posed earlier: In order to design a classifier that is able to predict classes on unseen data samples in a reliable fashion, where do we put the class boundaries? The answer to this problem is not so easy, and so we will spend some time on it. To simplify the treatment for the present, we restrict our attention to the two-class problem, often in two dimensions and sometimes in three. Our discussion will consider classification problems that can be solved by simple straight line separating boundaries as shown in Fig. 4.1. Towards the end of the chapter we will allow the boundaries to become piece-wise linear. As we show, simple binary threshold neurons and their networks can solve such straightforward classification problems. Increasing the number of classes complicates the problem considerably and is a problem that can be solved using non-linear or piecewise linear decision class boundaries generated by multilayer network architectures.

We formally restate the data classification problem for the two-class case with linear separating hyperplanes.

> *Given a set of Q data samples or patterns $\mathcal{X} = \{X_1, \ldots, X_Q\}, X_i \in \mathbb{R}^n$, drawn from two classes $\mathcal{C}_1$ and $\mathcal{C}_2$, find an appropriate hyperplane that separates the two classes so that the resulting classification decisions based on this class assignment are* on average *in close agreement with the actual outcome.*

Although this problem has been solved using a number of techniques, in this chapter we focus our attention on only the simplest neural network classifiers and attempt to provide an understanding of their geometrical behaviour. We specifically address the following question: How do we design single neuron dichotomizers, and what conditions, if any, need to be placed on the data set in order for these machines to classify data correctly? As it turns out, even the simplest threshold neuron has powerful properties that warrant careful mathematical characterization, and we now proceed to do the same. Threshold neurons are capable of classifying data sets that are *linearly separable*, and to this important property we turn our attention next.

See Ripley for a comprehensive overview of pattern classification techniques [477]. Bayesian classifiers base their decisions on probabilities calculated using Bayes Theorem [330]. Minimum distance classifiers place an arbitrary pattern X into category i associated with prototype points P_i in decision classes $\mathcal{C}_i$ such that $\|X - P_i\|$ is minimized, $i = 1, \ldots, C$ [136, 440, 509].

4.2 Convex Sets, Convex Hulls and Linear Separability

Recall that a vector of input features is called a pattern. The vector space from which valid patterns may be derived is then the pattern space, typically $\mathbb{R}^n$. In general any n-dimensional pattern can be represented by a point in pattern space, $\mathbb{R}^n$.

Consider two pattern sets $\mathcal{X}_1$ and $\mathcal{X}_2$ sampled from two classes $\mathcal{C}_1$ and $\mathcal{C}_2$. Many subsets of $\mathbb{R}^n$ contain the pattern sets $\mathcal{X}_1$ and $\mathcal{X}_2$. In the present context we are interested only in the smallest convex sets that contain these sets, and very shortly we will see why. But first the following definitions are in order.

Definition 4.2.1

Let $X, Y \in \mathcal{S} \subset \mathbb{R}^n$, then $\mathcal{S}$ is convex iff $\lambda X + (1-\lambda)Y \in \mathcal{S}$, $0 \leq \lambda \leq 1$, $\forall X, Y \in \mathcal{S}$. Equivalently, a set $\mathcal{S}$ is convex if it contains all points on all line segments with end points in $\mathcal{S}$.

In $\mathbb{R}^n$, spheres, ellipsoids and cubes are examples of convex sets. Toroids and disconnected sets are not convex. Figure 4.2 portrays some convex and non-convex sets.

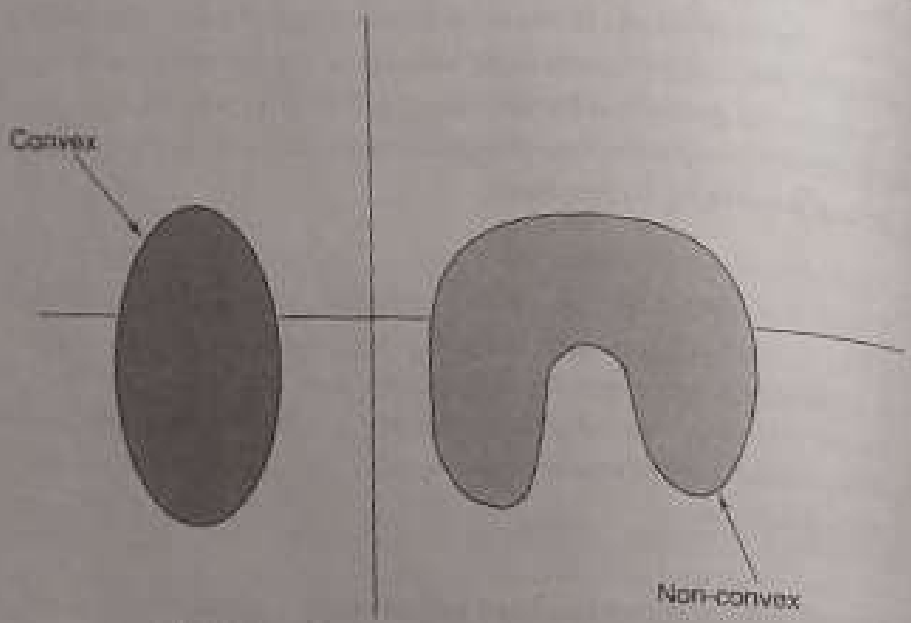


Fig. 4.2 Convex and non-convex sets in $\mathbb{R}^2$

Definition 4.2.2

The convex hull, $C(\mathcal{X}_i)$, of a pattern set $\mathcal{X}_i$ is the smallest convex set in $\mathbb{R}^n$ which contains the set $\mathcal{X}_i$. Equivalently, consider every convex set $\mathcal{S}_\alpha$ such that $\mathcal{X}_i \subset \mathcal{S}_\alpha \subset \mathbb{R}^n$, $\alpha \in \mathcal{I}$, where $\mathcal{I}$ is an index set. Then the convex hull of $\mathcal{X}_i$, $C(\mathcal{X}_i) = \bigcap_{\alpha \in \mathcal{I}} \mathcal{S}_\alpha$.

This only means that we need to take the intersection of all $\mathbb{R}^n$ subsets that contain the pattern set in question. This intersection yields the smallest convex set in $\mathbb{R}^n$ that contains the pattern set. Figure 4.3 shows a computer

generated convex hull of the Iris data [12], using MATLAB. Notice how the convex hull of *iris sestosa* is disjoint from the convex hulls of *iris versicolor* and *iris virginica*. However, the convex hulls of *iris versicolor* and *iris virginica* are not separable.

The problem of convex hull computation is an interesting area of research. Neural networks have been applied to this problem (see [432]).

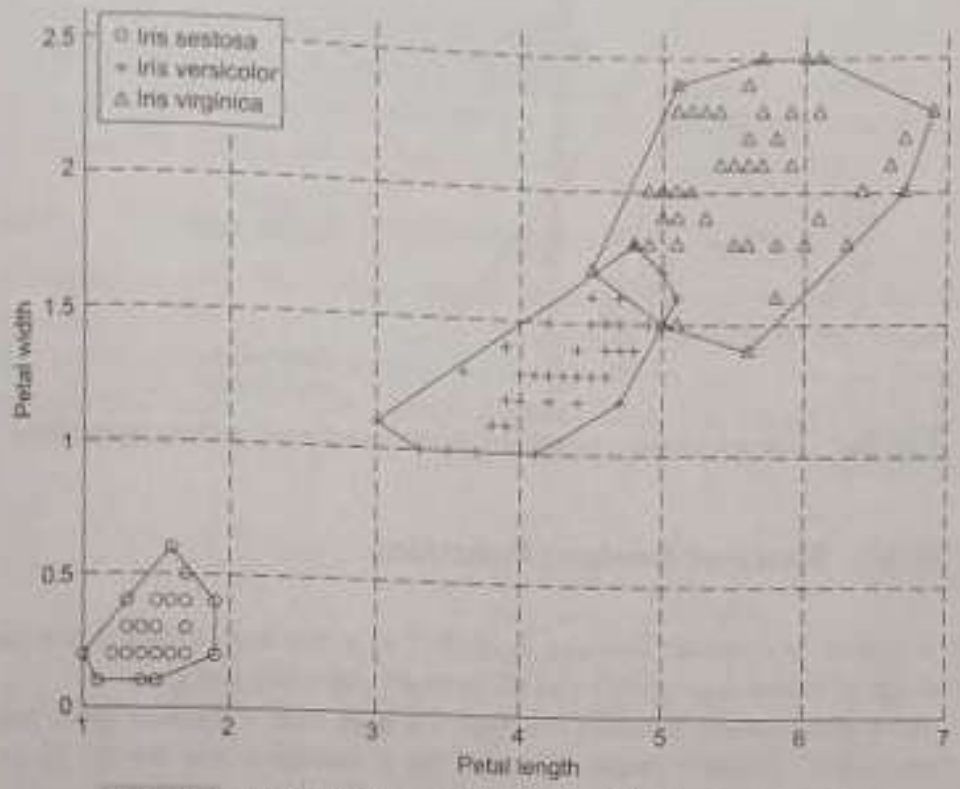


Fig. 4.3 MATLAB generated convex hulls for Iris data [12]

Note: This is a view of the 4-dimensional Iris data projected on to the x_3–x_4 axes. The convex hull was generated using MATLAB's convhull command.

It should be intuitively clear that if the convex hulls of two pattern sets are non-overlapping, we can define a separating hyperplane that slices the pattern space into two halves, such that the pattern sets are separated. This leads to the concept of linear separability.

Definition 4.2.3

Two pattern sets $\mathcal{X}_i$ and $\mathcal{X}_j$ are said to be *linearly separable* if their convex hulls are disjoint, that is if $C(\mathcal{X}_i) \cap C(\mathcal{X}_j) = \phi$.

As shown in Fig. 4.4 since some minimum distance separates the two convex hulls, there exists a hyperplane Π, that separates the two sets. In the figures, we use the shorthand C_i to denote the convex hull $C(\mathcal{X}_i)$. The convex hull C_i is representative of the pattern class $\mathcal{C}_i$.

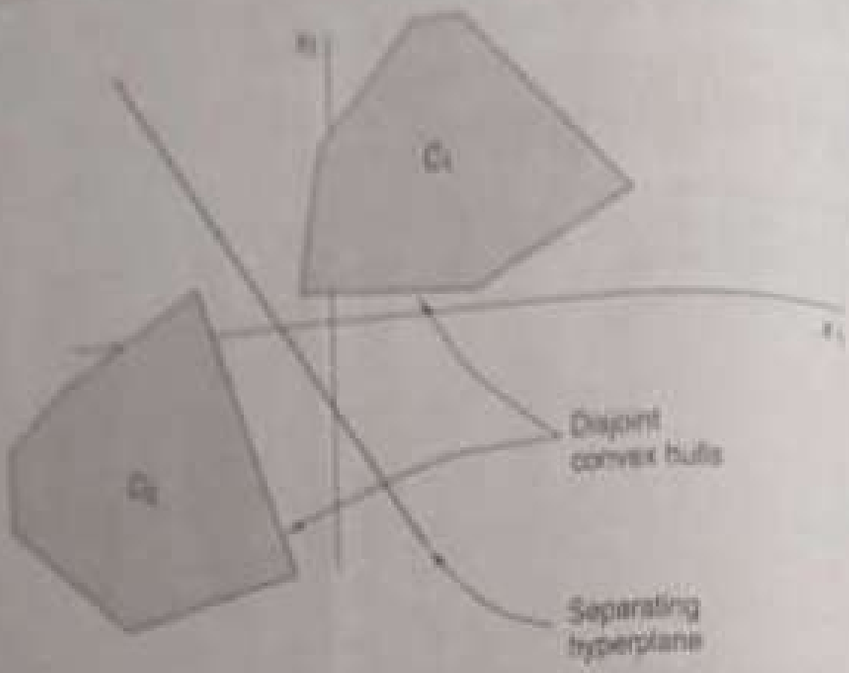


Fig. 4.4 Linearly separable pattern classes and a separating hyperplane

4.3 Space of Boolean Functions

It is useful to consider Boolean functions as a working example for the concept of linear separability and elementary classifier design.

An n-dimensional Boolean function is a map from a domain space that comprises 2^n possible combinations of the n variables into the set $\{0, 1\}$. Geometrically, it is instructive to think of these 2^n domain points as corners of an n-dimensional hypercube. For example, in two dimensions, the set of possible inputs is $\{00, 01, 10, 11\}$. We denote precisely these four corners as the Boolean 2-cube: $\mathbb{B}^2 = \{0, 1\} \times \{0, 1\} = \{0, 1\}^2 = \{00, 01, 10, 11\}$. These four points form the corners of the unit square $\mathbb{I}^2 = [0, 1] \times [0, 1] = [0, 1]^2$ in $\mathbb{R}^2$. Similarly, in three dimensions there are eight input combinations that comprise the Boolean 3-cube $\mathbb{B}^3$.

$[0, 1] \times [0, 1]$ represents the Cartesian product of the two closed unit intervals. In general, I^n is sometimes referred to as the *fuzzy n-cube* [305]. We will have opportunity to revisit this in Chapter 14.

In $\mathbb{B}^n$, there are 2^n points corresponding to each of the 2^n possible combinations of n variables. In a *Boolean function*, each of these domain points maps to either 0 or 1, $f : \mathbb{B}^n \to \{0, 1\}$. For the present discussion, we denote the set of points that map to 0 as $\mathcal{X}_0$, and the set of those that map to 1 as $\mathcal{X}_1$. Put another way, the sets $\mathcal{X}_0, \mathcal{X}_1$ comprise points that are pre-images of range points 0, 1: $\mathcal{X}_0 = f^{-1}(0)$ and $\mathcal{X}_1 = f^{-1}(1)$. Each unique assignment of 0-1 values to the 2^n possible inputs in n-dimensions represents a Boolean function. Therefore, in n-dimensions, there are 2^{2^n} such unique assignments that can be made, and thus 2^{2^n} possible functions.

If there exists a hyperplane that separates $\mathcal{X}_0$ from $\mathcal{X}_1$, then they are linearly separable. The hyperplane should not pass through any of the points of $\mathbb{B}^n$ since that would result in an ambiguous classification for the point in question. In terms of Boolean functions we therefore have the following alternative but equivalent definition for linear separability.

Definition 4.3.1

A Boolean function $f(x_1, \ldots, x_n) : \mathbb{B}^n \to \{0, 1\}$, is linearly separable if there exists a hyperplane Π in $\mathbb{R}^n$ that strictly separates $\mathcal{X}_0$ from $\mathcal{X}_1$, and $\Pi \cap \{0, 1\}^n = \phi$.

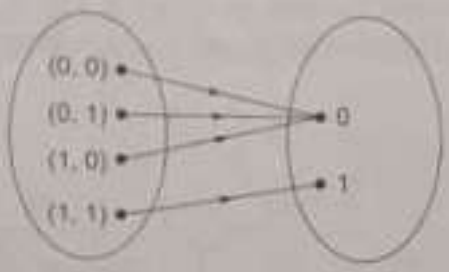
Of the 2^{2^n} possible Boolean functions, some are linearly separable and some are not. Examples of functions that are not linearly separable are the exclusive OR and the exclusive NOR functions.

4.3.1 Example: Two Dimensional Logical AND Function

Consider the two dimensional logical AND function. Assuming domain Boolean variables x_1 and x_2, Fig. 4.5(a) shows the truth table of the function. Figure 4.5(b) depicts the function $f_\wedge(x_1, x_2) : \mathbb{B}^2 \to \{0, 1\}$ as a map from a domain of 2^2 points ($n = 2$) which comprise $\mathbb{B}^2$, into the range set $\{0, 1\}$. For $f_\wedge(\cdot)$, $\mathcal{X}_0 = f_\wedge^{-1}(0) = \{(0, 0), (0, 1), (1, 0)\}$ and $\mathcal{X}_1 = f_\wedge^{-1}(1) = \{(1, 1)\}$.

x_1	x_2	$f_\wedge$
0	0	0
0	1	0
1	0	0
1	1	1

(a)



(b)

Fig. 4.5 The Boolean AND function $f_\wedge : \mathbb{B}^2 \to \{0, 1\}$

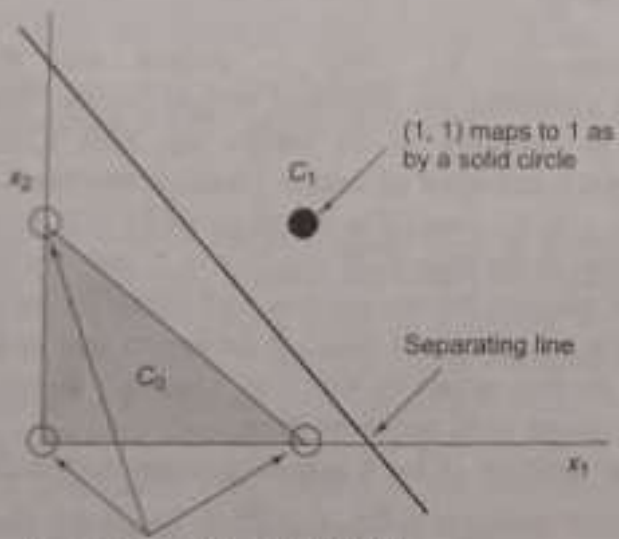


Fig. 4.6 The geometry of the Boolean AND function

The AND function is linearly separable because the convex hulls $C_0 = C(\mathcal{X}_0)$ (the shaded triangle with corners at (0, 0), (0, 1), and (1, 0) in Fig. 4.6) and $C_1 = C(\mathcal{X}_1)$ (which is the singleton set $\{(1, 1)\}$), are disjoint.

It is convenient and instructive to view the problem geometrically, as shown in Fig. 4.6. Here the four domain points comprising $\mathbb{B}^2$ have been marked on the real plane $\mathbb{R}^2$, with unfilled circles corresponding to points that map to a 0 and a solid circle corresponding to points that map to a 1. Referring further to Fig. 4.6, note that the convex hulls $C_0 = CH(X_0)$ (the shaded triangle with corners at (0,0), (0,1), and (1,0)) and $C_1 = CH(X_1)$ (the singleton set $\{(1, 1)\}$), are disjoint. Clearly, one can draw many straight lines that separate the solid circle from the unfilled circles. Figure 4.6 shows one such possibility, and $f_n(\cdot)$ is therefore a linearly separable function.

4.4 Binary Neurons are Pattern Dichotomizers

The simple binary (or bipolar) threshold neuron, often referred to as a McCulloch-Pitts neuron or a threshold logic neuron (TLN), can classify linearly separable pattern classes. To see this, we return to the two dimensional case.

Consider the neuron shown in Fig. 4.7, where for clarity the activation aggregation and the signal function have been separated. As before, vector

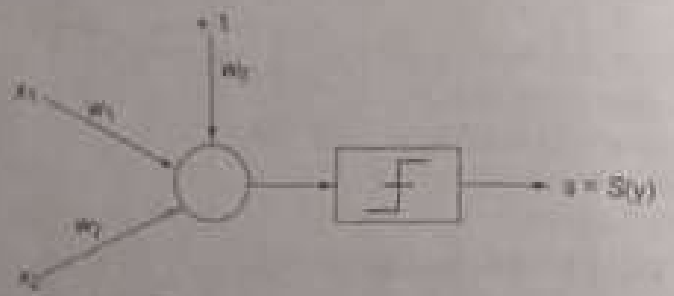


Fig. 4.7 A binary threshold logic neuron

$X = (1, x_1, x_2)$ represents the input to neuron and the vector $W = (w_0, w_1, w_2)$ represents the neuronal weight vector. The internal bias or threshold is modelled by the weight w_0, with a constant +1 input. The weighted summation of inputs yields a neuronal activation, $y = X^t W = w_1 x_1 + w_2 x_2 + w_0$. The function $y(X) = 0$ is called the *discriminant function* of the neuron. If $y > 0$, $s = 1$, and if $y < 0$, $s = 0$. Given a fixed set of weights, the neuron fires a +1 signal for inputs (x_1, x_2) which yield positive activation and fires a 0 for inputs that yield negative activation. Inputs are thus separated into two "classes". Inputs satisfying the discriminant function yield zero activation.

The discriminant function is essentially the locus of all points in pattern space that yield zero activation.

The discriminant function $y(\cdot) = 0$ is readily recast into the form:

$$x_2 = -\frac{w_1}{w_2} x_1 - \frac{w_0}{w_2} \tag{4.1}$$

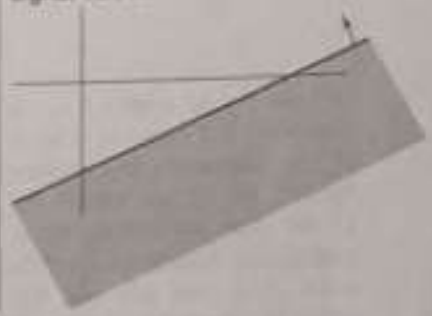
which is simply the equation of a straight line in the x_1–x_2 plane. The neuron discriminant function thus represents a straight line in the two dimensional pattern space! Notice that the slope and intercept of the straight

Comparing this with the standard equation of a straight line, $y = mx + c$, we see that the weight ratio $-w_1/w_2$ decides the slope of the line, and the ratio $-w_0/w_2$, its intercept.

line are functions of only the weights. In other words, the weights decide the placement of the line in the plane $\mathbb{R}^2$. This straight line slices the plane into two halves: one half where patterns cause the neuron to generate positive inner products and thus fire a +1, and the other half where pattern space points cause the neuron to generate negative inner products and thus fire a 0. Binary neurons are pattern dichotomizers that generate two decision regions in the pattern space (see margin figure and note).

It follows therefore that if a pattern set is linearly separable, then it should be possible to design the neuronal weights so that the resulting discriminant function provides the appropriate classification boundary. We illustrate this design for the Boolean AND classification problem.

Notice that for the given choice of weights the hyperplane has an *orientation* such that points above the line yield positive activations (and thus generate a +1 signal) whereas points below the line generate negative activations (and thus generate a signal of 0). The orientation is shown in Fig. 4.8 by the small arrow on the straight line indicating the half plane that translates to a +1 signal value.



4.4.1 Example: Geometrical Design of AND Classifier

Shown in Fig. 4.8 is the now familiar geometrical representation of the Boolean AND function.

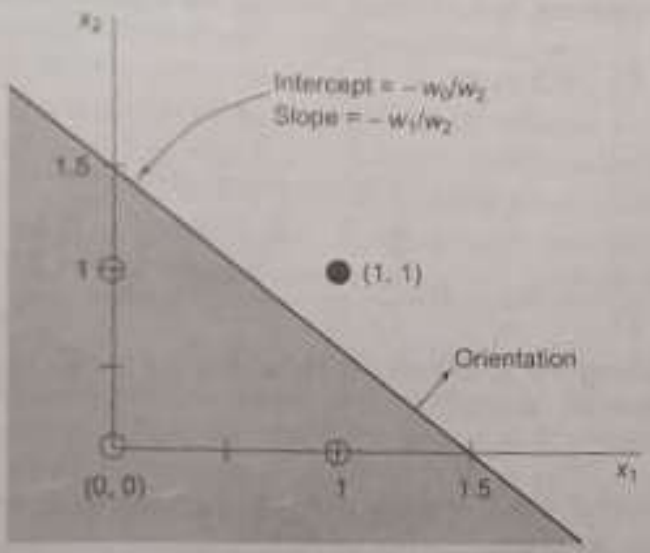


Fig. 4.8 The geometry of the Boolean AND function with slopes and intercepts in terms of neuron weights

Since this function is linearly separable it is straightforward to find a separating straight line. Figure 4.8 shows one such placement with slope $m = -w_1/w_2 = -1$, and intercept $c = -w_0/w_2 = 1.5$. Choosing $w_1 = w_2 = 1$ and $w_0 = -1.5$ yields a neuron classifier that appropriately partitions the pattern space for AND classification. Here, the value of the threshold is -1.5.

Table 4.1 summarizes the values of the activity q that aggregates from external inputs ($q = w_1x_1 + w_2x_2$) for each input. It also shows the corresponding net neuronal activation $y = q + w_0$ and the signal s. If we look at the design problem in the s–q plane then Table 4.1 shows that different input combinations generate values of $q = 0, 1, 2$ given the

Table 4.1 AND neuron activations and signals for different inputs

x_1	x_2	$q = w_1x_1 + w_2x_2$ ($w_1 = w_2 = 1$)	$y = q - 1.5$	s
0	0	0	−1.5	0
0	1	1	−0.5	0
1	0	1	−0.5	0
1	1	2	0.5	1

weights $w_1 = w_2 = 1$. We wish the neuronal signal s should be 1 on if $q = 2$ and zero otherwise. Therefore, we can place the signal functi anywhere in the interval (1, 2) and get the appropriate result. As Fig. 4 shows, our design placed the signal function at 1.5 ($\theta = -1.5$), midwa through the interval. In fact any position between $1 + \epsilon$ and $2 - \epsilon$ woul suffice. It is important to realize that moving the signal function in the $x-$

The reason for saying $2 - \epsilon$ is that at $x_1 = 2$ (with $w_1 = w_2 = 1$), the separating line passes through (1,1). This should not be the case for it leads to an ambiguous classification for the point (1,1). You can see this from Fig. 4.9 since $x_1 = 2$ would place the signal function at 2 on the q axis. Similar reasoning results in the lower limit being $1 + \epsilon$.

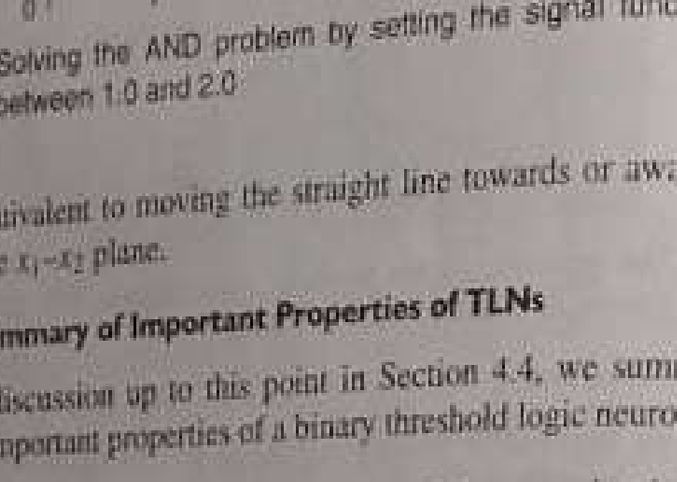


Fig. 4.9 Solving the AND problem by setting the signal function shift between 1.0 and 2.0

plane is equivalent to moving the straight line towards or away from the origin in the x_1–x_2 plane.

4.4.2 Summary of Important Properties of TLNs

From our discussion up to this point in Section 4.4, we summarize the following important properties of a binary threshold logic neuron.

- A threshold logic neuron employs a single inner product based linear discriminant function $y : \mathbb{R}^{n+1} \to \mathbb{R}$.

$$y(X) = X^T W \tag{4.2}$$

where $X, W \in \mathbb{R}^{n+1}$ and the bias or threshold value w_0 is included into the weight vector.

- The hyperplane *decision surface* $y(X) = 0$ divides the space into two regions, one of which the TLN assigns to class $\mathcal{C}_0$ and the other to class $\mathcal{C}_1$. We say that the TLN dichotomizes patterns by a hyperplane decision surface in $\mathbb{R}^n$.

- The orientation of the hyperplane is determined by the weights $w_1, \ldots, w_n$.
- The position of the hyperplane is proportional to w_0.
- The distance from the hyperplane to an arbitrary point $X \in \mathbb{R}^n$ is proportional to the value of $y(X)$.
- A pattern classifier which employs linear discriminant functions is called a *linear machine*.

We now extend the above ideas of linear separability to the more general C-class problem.

4.4.3 Linear Classifications of Multi-class Patterns

In the more general C class problem we consider a finite set $\mathcal{X}$ of Q distinct patterns $\{X_1, \ldots, X_Q\}$, and assume that the patterns in $\mathcal{X}$ are classified in such a manner that each pattern in $\mathcal{X}$ belongs to exactly one of the C classes. This classification divides $\mathcal{X}$ into the subsets $\mathcal{X}_1, \ldots, \mathcal{X}_C$ such that each pattern in $\mathcal{X}_i$ belongs to class $\mathcal{C}_i$ for $i = 1, \ldots, C$.

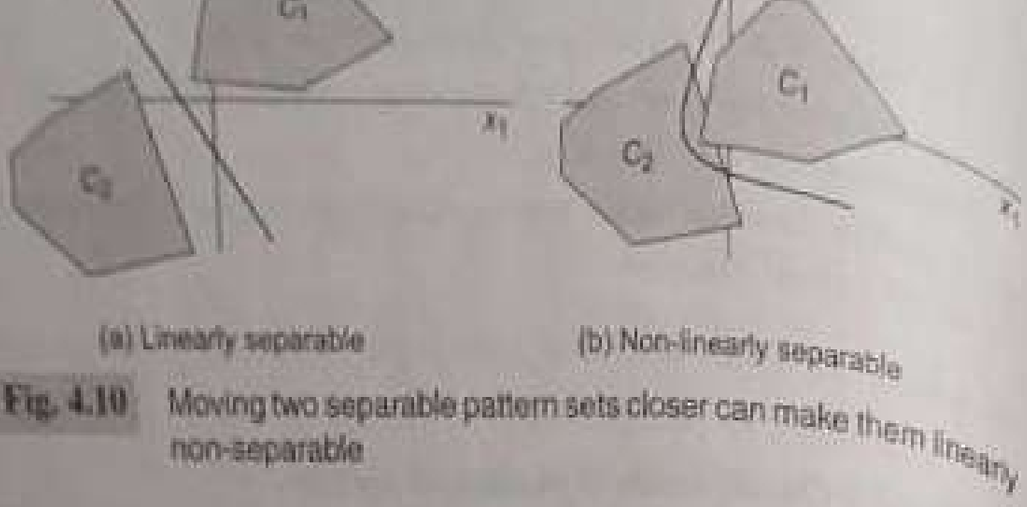
If a linear machine can categorize each of the patterns in $\mathcal{X}$ correctly, we say that the classification of $\mathcal{X}$ is a *linear* classification and that the subsets $\mathcal{X}_1, \ldots, \mathcal{X}_C$ are *linearly separable*. Alternatively, a classification of $\mathcal{X}$ is linear and the subsets $\mathcal{X}_1, \ldots, \mathcal{X}_C$ are linearly separable if and only if there exist linear discriminant functions $y_1, \ldots, y_C$ such that $y_i(X) > y_j(X)\ \forall X \in \mathcal{X}_i,\ i, j = 1, \ldots, C;\ j \neq i$.

Consider the special case of the above definition with $C = 2$. Here the machine would have two discriminant functions $y_1(X)$ and $y_2(X)$. We then say that a *dichotomy* of $\mathcal{X}$ into two subsets $\mathcal{X}_1$ and $\mathcal{X}_2$ is a *linear dichotomy* if and only if a linear discriminant function $y(X) = y_1(X) - y_2(X)$ exists such that $y(X) > 0\ \forall X \in \mathcal{X}_1$ and $y(X) < 0\ \forall\ X \in \mathcal{X}_2$. Clearly $\mathcal{X}_1$ and $\mathcal{X}_2$ are linearly separable if and only if a hyperplane exists which has each member of $\mathcal{X}_1$ on one side and each member of $\mathcal{X}_2$ on the other side.

We can generalize the notion of linear separability further by noting that the decision regions of a linear machine are convex. It follows that if the subsets $\mathcal{X}_1, \ldots, \mathcal{X}_C$ are linearly separable, then they are pair-wise linearly separable.

4.5 Non-linearly Separable Problems

Our study of the binary threshold neuron was formulated in the context of two dimensions. In general a neuron with n inputs would have a discriminant function representing an $n - 1$ dimensional hyperplane in $\mathbb{R}^n$, which would slice the entire $\mathbb{R}^n$ into two parts: one where the neuron output would be a +1, and the other where the output would be a 0. The non-linear element of this model is represented by a hard limiting step function at its output.



(a) Linearly separable (b) Non-linearly separable

Fig. 4.10 Moving two separable pattern sets closer can make them linearly non-separable

Linear separability requires that the patterns to be classified be sufficiently separated from one another to ensure decision surfaces comprise only of hyperplanes. For example, the two convex hulls C_1, C_2 of the two pattern sets in Fig. 4.10(a) are separable. If they are moved closer, a point comes when the convex hulls overlap, (Fig. 4.10(b)) and no straight line can separate the two pattern sets. The pattern sets become *linearly non-separable* but may be *non-linearly separable* as shown in Fig. 4.10(b). In short, a single neuron can no longer classify the two classes effectively, and errors in classification will result when we try to use a binary threshold neuron classifier to separate the patterns.

4.5.1 XOR is Not Linearly Separable

The Boolean XOR function $f_{\oplus} : \mathbb{B}^n \rightarrow \{0, 1\}$ is an example of a non-linearly separable problem which we study in detail. This is shown geometrically in Fig. 4.11 for the two dimensional case.

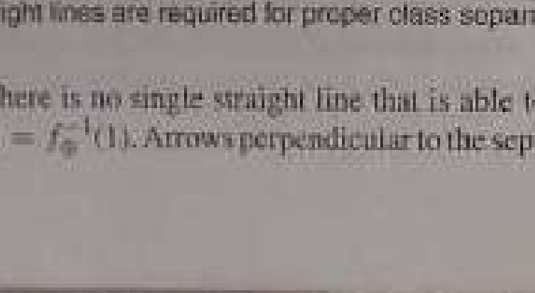


Fig. 4.11 The geometry of the Boolean XOR function shows that two straight lines are required for proper class separation

Notice that there is no single straight line that is able to separate $X_0 = f_{\oplus}^{-1}(0)$ from $X_1 = f_{\oplus}^{-1}(1)$. Arrows perpendicular to the separating lines that

indicate the line orientations, demarcate a central region where the classifier output should be a 1. Clearly, one threshold logic neuron cannot do the job. This can be proved mathematically as Theorem 4.5.1 shows.

Theorem 4.5.1

No threshold logic neuron exists that can solve the logical XOR classification problem, $x_1 \oplus x_2$.

Proof By contradiction. We assume a TLN with weights w_0, w_1, w_2 and inputs x_1, x_2. For the XOR function, $x_1 \oplus x_2 = 1$ iff $w_1x_1 + w_2x_2 + w_0 \geq 0$. Since the XOR operation is symmetric, we must also have $x_2 \oplus x_1 = 1$ iff $w_1x_2 + w_2x_1 + w_0 \geq 0$. These two conditions imply that $\frac{(w_1+w_2)}{2}x_1 + \frac{(w_1+w_2)}{2}x_2 + w_0 \geq 0$. This implies that $wx + w_0 \geq 0$ where $w = \frac{(w_1+w_2)}{2}$ and $x = x_1 + x_2$. Notice that $wx + w_0$ is a first degree polynomial in x, which must be less than zero for $x = 0$ $(0 \oplus 0)$, greater than zero for $x = 1$ $(0 \oplus 1$ and $1 \oplus 0)$, and less than zero for $x = 2$ $(1 \oplus 1)$. This is impossible since a first degree polynomial is monotonic and therefore cannot change sign more than once. We thus conclude that there is no such polynomial, and therefore there is no TLN that can solve the XOR problem.

Observe that the XOR classification problem is equivalent to the simple modulo-2 addition: $x_1 \oplus x_2$. In the proof alongside: "iff" denotes "if and only if".

So how do we solve such a problem?

4.5.2 Solving the XOR Problem

Since the desired central region of 1's in the geometry is created by two straight lines, a solution to the problem emerges if we first implement each separating line as the discriminant function of a TLN, and then take the logical intersection of their outputs.

Figures 4.12(a) and (b) provide a three-dimensional perspective of what we can construct with a single binary neuron. Our points of interest, $\mathbb{B}^2$, lie on the x_1–x_2 plane and the z-axis plots the neuronal output for specific values of x_1 and x_2. Notice that the neuronal output changes abruptly from 0 to 1 at the separating line that represents the neuron's discriminant function. In Fig. 4.12(a) which portrays the logical OR function, the neuron fires 0 to the left of the separating line, and 1 to the right of the separating line. In Fig. 4.12(b) the neuron fires a 1 to the left of the line, and a 0 to the right of the line. This function represents a logical NAND function since the output, s, is 0 for (1,1) and 1 for (0,0), (0,1) and (1,0). Single binary neurons can do no better than generate functions such as these. However, combining the results of Figs 4.12(a) and (b) using a logical AND (which can be implemented by a third binary neuron), results in the response surface portrayed in Fig. 4.12(c). Figure 4.12(c) in fact represents the logical XOR function which we see is readily constructed by taking the logical AND of the OR and NAND functions respectively: $x_1 \oplus x_2 = (x_1 + x_2) \cdot \overline{x_1 x_2} = x_1\bar{x}_2 + \bar{x}_1 x_2$.

Figure 4.13(a) shows the architecture of a neural network that can implement the XOR logic. This is a two layer architecture where Layer 1 implements the logical OR and NAND functions (neurons 1,2 respectively)

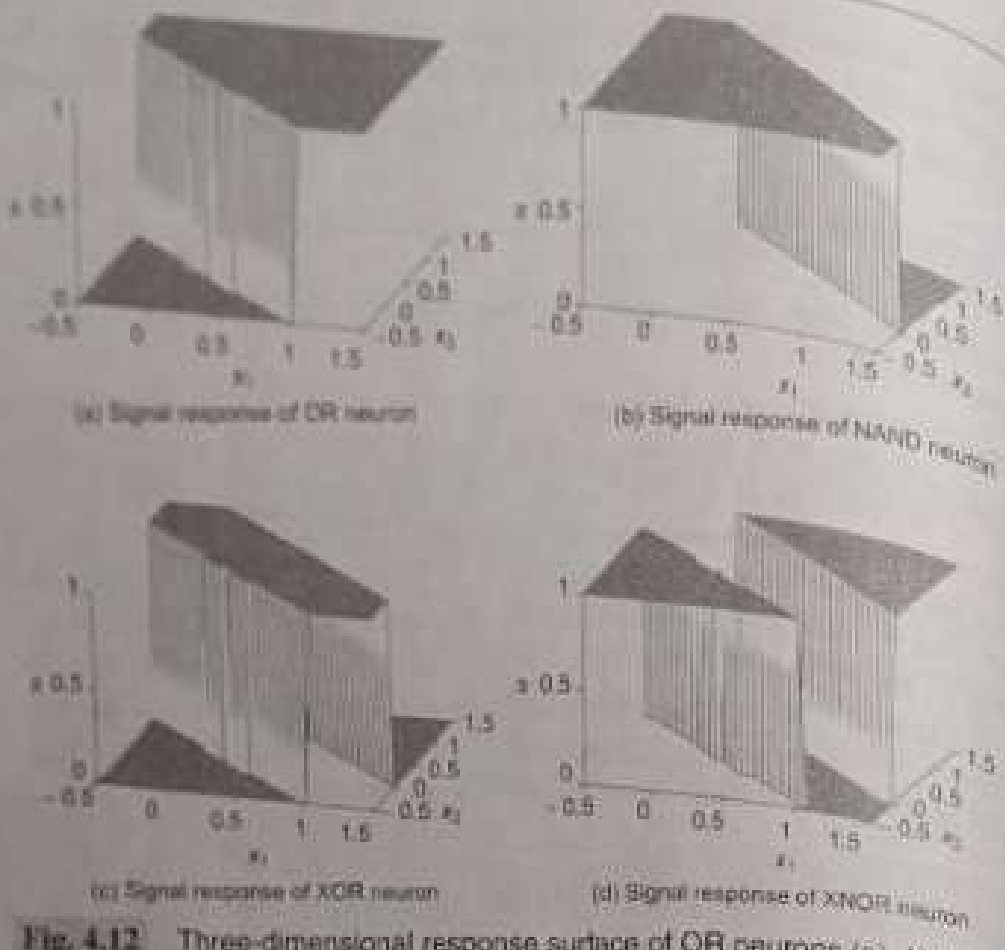


Fig. 4.12 Three-dimensional response surface of OR neurons (a), NAND neurons (b), The logical AND of OR-NAND neuron responses generates an XOR neuron (c). 3-dimensional response surface of the XNOR neuron (d)

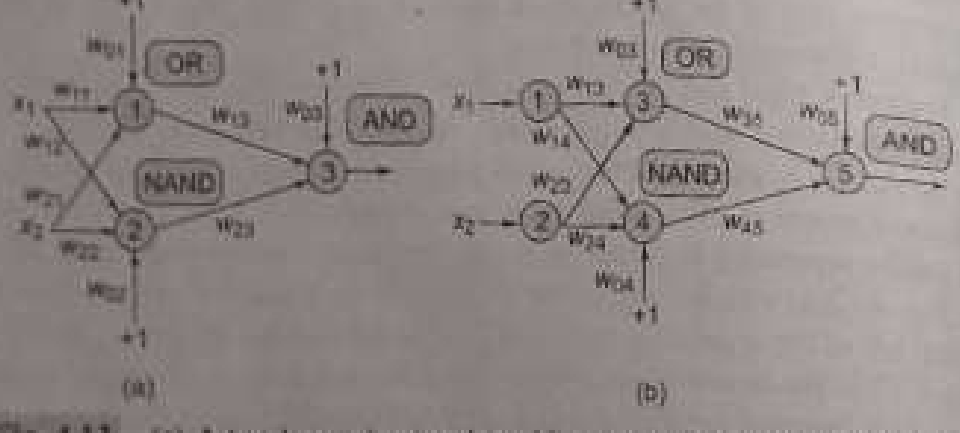


Fig. 4.13 (a) A two-layered network architecture to implement the XOR function. (b) A more common three-layered version of the same network with linear input units shown explicitly

and Layer 2 takes the logical AND (neuron 3) of the resulting outputs of Layer 1 to generate the XOR function. We already know how to design the weights for each of these neurons, and thus the solution to the XOR problem is complete. Figure 4.13(b) shows the more common notation for portraying such a multilayer network. Here, external inputs go to *linear* input neurons

(1 and 2) which simply transmit these inputs without any modification. The middle layer (comprising neurons 3,4) is now theoretically isolated from the inputs and is called the *hidden layer*. The output neuron (5) belongs to the *output layer*. The neurons in Fig. 4.13(b) have been re-numbered for assignment of proper labels to the weights.

4.6 Capacity of a Simple Threshold Logic Neuron

We have seen that a simple TLN implements a specific class of functions from $\mathbb{R}^n$ into the set $\{0, 1\}$, namely, the class of linearly separable functions. A more general question now comes to mind. If we were to arbitrarily assign image points 0 or 1 to some p domain points, how many such random assignments could we expect the TLN to successfully implement. We call the number of such assignments that the TLN can correctly implement, its *capacity*. In other words, what is the maximum capacity of a TLN? Intuition tells us that this capacity of TLNs should obviously depend on the linear separability of the pattern set. For the present, we consider a TLN with n continuous valued inputs and a signal function that uses the deterministic threshold limit $s \in \{0, 1\}$. The extension to the multineuron case is trivial since output neurons and their connections are independent.

How many random input–output pairs can we expect to *store* reliably in a threshold logic neuron (TLN)?

4.6.1 Linear Dichotomies of p Points in n Dimensions

An understanding of the capacity of a TLN is facilitated by the notion of the number of functions that it can compute. Throughout we consider the case of functions $f : \mathbb{R}^n \to \mathbb{B}$. Recall that a TLN with n inputs essentially implements a linear discriminant function (an $(n-1)$–dimensional hyperplane in n–dimensions) which creates a linear dichotomy. Its classification capability must therefore depend not only on the dimension itself, but also on the number of points (patterns) that are to be classified, and their relative positions in space. What we really want to know therefore is the number of linear dichotomies that can be induced on p points in n–dimensions.

When we say n inputs we mean n external inputs. The pattern space is actually augmented by 1 to include the bias.

Points in General Position

Denote the number of linear dichotomies that can be induced on p points in n–dimensional space as $L(p, n)$. Then $L(p, n)$ is equal to twice the number of ways in which p points can be partitioned by an $(n-1)$–dimensional hyperplane, since for each distinct partition, there are two different classifications. Here we assume that no three points in space are collinear, since this would effectively reduce the number of inducible dichotomies. To formalize this notion we require the following definition.

Definition 4.6.1

For $p > n$, we say that a set of p points is in *general position* in a n-dimensional space if and only if no subset of $n+1$ points lie on an $(n-1)$-dimensional hyperplane. When $p \leq n$, a set of p points is in general position if no $(n-2)$-dimensional hyperplane contains the set.

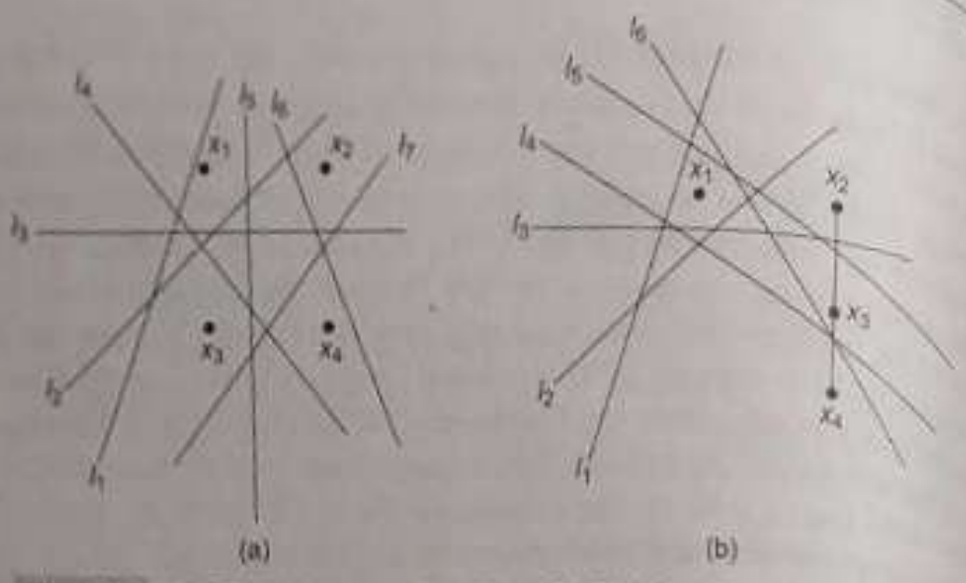


Fig. 4.14 (a) Points in general position; (b) Points not in general position

For example, the four points in Fig. 4.14(a) are in general position, whereas those in Fig. 4.14(b) are not. In Fig. 4.14(a) the lines $l_1, \ldots, l_7$ implement all possible *linear partitions* of these four points. Consider the line l_3 in Fig. 4.14(b) in particular which could be a decision surface that implements either one of the following classifications: $\{X_1, X_2\} \in \mathcal{C}_0$ and $\{X_3, X_4\} \in \mathcal{C}_1$; or $\{X_1, X_2\} \in \mathcal{C}_1$ and $\{X_3, X_4\} \in \mathcal{C}_0$.

$\mathcal{C}_0$ is the class identified by a TLN signal 0; $\mathcal{C}_1$ is the class identified by a TLN signal 1.

Henceforth we assume that the p points under consideration are in general position. Observe from Fig. 4.14(a) that $L(4, 2) = 2 \times 7 = 14$ which represents some kind of a capacity of a single TLN to map functions of four domain points in two dimensions. By way of example, note that in the special case $f : \mathbb{B}^2 \to \{0, 1\}$ which we studied, the number of possible functions was 16, and two of them (XOR and XNOR) were not computable by a single TLN. Notice finally, that the exact configuration of the points does not influence the number of linear partitions unless three of the points are collinear as shown in Fig. 4.14(b) where there are only six linear partitions.

Recall that each TLN effectively divides an n-dimensional input space into two regions separated by an $(n-1)$-dimensional hyperplane. (For the case of zero bias ($w_0 = 0$) this hyperplane goes through the origin.) All points on one side of this hyperplane generate a positive inner product and yield a signal 1; all those on the other side yield a signal 0.

We now concentrate on the following question: Given a set of p random points in n-dimensional space, for how many of the 2^p possible {0, 1} assignments can we find a separating hyperplane that divides one class from another? The answer is precisely $L(p, n)$.

Each of p points in the domain space can be assigned an image of either 0 or 1 in the range space. This gives a total of 2^p possible assignments.

Computing the Dichotomies

One can calculate $L(p, n)$ using induction. We assume that we know the the number of dichotomies $L(p, n)$ of p points in n dimensions, and calculate the number of dichotomies generated when we add a single new point P in n-dimensions.

It is easy to see that some of the old dichotomies can be drawn through P without changing their classification structure, that is, their 0–1 assignments. Call the number of such dichotomies L_P. Each of these old dichotomies will generate two new dichotomies: one in which P is assigned a 0, and the other in which P is assigned a 1. These new dichotomies are generated by perturbing the hyperplane drawn through P to include P on one side or the other. The dichotomies (those that cannot be drawn through P without changing the original assignments) are then $L(p, n) - L_P$. We therefore have

$$L(p+1, n) = L(p, n) - L_P + 2L_P = L(p, n) + L_P \qquad (4.3)$$

L_P is straightforward to compute. As Fig. 4.15 shows for the 2-dimensional case, projection of each of the p points and the L_P hyperplanes (constrained to go through P) onto a subspace of dimension $n-1$ leaves the number of dichotomies unchanged! This means that the addition of the constraint that the hyperplanes must pass through the point P makes the problem essentially an $(n-1)$–dimensional one. Therefore, $L_P = L(p, n-1)$.

Equation 4.3 thus yields the following recursive relation that allows us to calculate the total number of dichotomies:

$$L(p+1, n) = L(p, n) + L(p, n-1) \qquad (4.4)$$

Equation 4.4 can be recursively solved from $p, \ldots, 1$. For example, $L(4, 3) = \binom{3}{0}L(1, 3) + \binom{3}{1}L(1, 2) + \binom{3}{2}L(1, 1) + \binom{3}{3}L(1, 0) = 16$, where we have to use the boundary condition: $L(1, n) = 2, n \geq 0$.

$L(1, 0) = 2$ is mathematically sound. A single point (which is the entire space itself) has zero dimension, and can be assigned an image 0 or 1.

In general, we have,

$$L(p, n) = \binom{p-1}{0}L(1, n) + \binom{p-1}{1}L(1, n-1) + \ldots + \binom{p-1}{p-1}L(1, n-p+1) \qquad (4.5)$$

$$= \sum_{i=0}^{p-1}\binom{p-1}{i}L(1, n-i) \qquad (4.6)$$

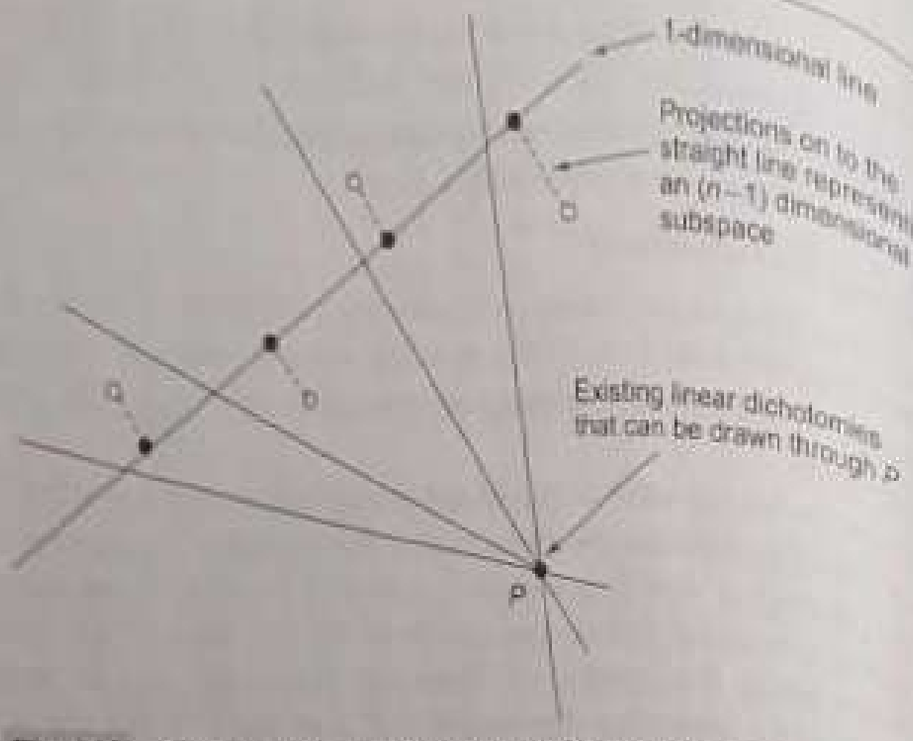


Fig. 4.15 Projection of L_P dichotomies (those that can be drawn through l' without changing the original assignments on p points) onto a lower dimensional subspace (1–dimensional in this case) yields an $(n-1)$–dimensional problem with $L_P = L(p, n-1)$

For $p > n+1$ the second argument of L becomes negative in some terms, but these terms can be eliminated by assuming $L(1, i) = 0, i < 0$, which is precisely what we have done in Eq. 4.7.

However, if $p > n+1$, Eq. 4.6 reduces to Eq. 4.7.

$$L(p, n) = 2\sum_{i=0}^{n}\binom{p-1}{i} \tag{4.7}$$

Alternatively, if $p \leq n+1$, Eq. 4.6 can be recast as

$$L(p, n) = 2\sum_{i=0}^{p-1}\binom{p-1}{i} = 2 \cdot 2^{p-1} = 2^p \tag{4.8}$$

where we use the standard result that $\binom{p}{q} = 0,\ p < q$. Equations 4.7 and 4.8 can be combined and written compactly as shown in Eq. 4.9.

$$L(p, n) = 2\sum_{i=0}^{\min(n,\,p-1)}\binom{p-1}{i} \tag{4.9}$$

4.6.2 A Probabilistic Estimate of Computable Dichotomies

With p points in n–dimensions there are 2^p possible functions. The probability that they are implementable by a TLN is then directly the ratio of $L(p, n)$ and 2^p. We therefore have the probability $P(p, n)$ that a dichotomy defined on p points in n–dimensions is computable by a TLN as

$$P(p, n) = \frac{L(p, n)}{2^p} = \begin{cases} 2^{1-p}\sum_{i=0}^{n}\binom{p-1}{i} & \text{for } p > n+1 \\ 1 & \text{for } p \leq n+1 \end{cases} \tag{4.10}$$

Equation 4.10 is plotted in Fig. 4.16 where λ expresses p as a linear function of the dimension: $p = \lambda(n+1)$.

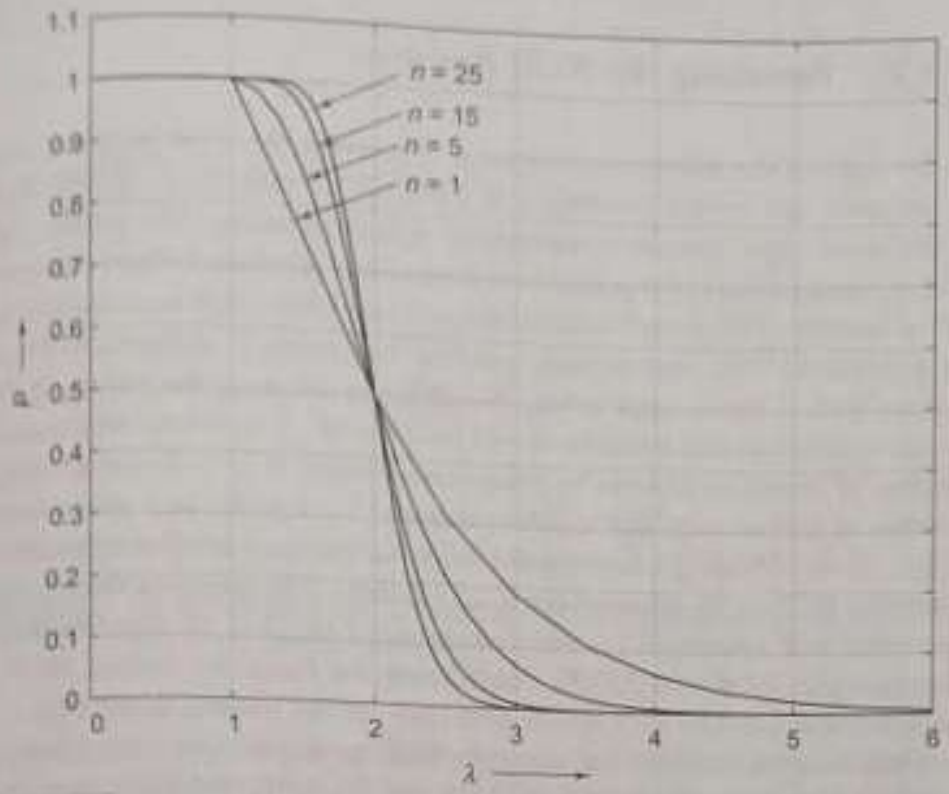


Fig. 4.16 A plot of P vs λ plotted in any dimension, always goes through the point 1/2

Also notice that when $p = 2(n+1)$, that is $\lambda = 2$,

$$P(2(n+1), n) = \frac{1}{2} \tag{4.11}$$

and so the entire family of probability curves go through 1/2 at $\lambda = 2$. For large n, notice the sharp decline from probability 1 to 0 around $\lambda = 2$. In fact as $n \to \infty$ the probability curve tends to a binary threshold function centered at $\lambda = 2$. This notion is captured by the following limits:

$$\lim_{n\to\infty} P((2+\epsilon)(n+1), n) = 0 \tag{4.12}$$

$$\lim_{n\to\infty} P((2-\epsilon)(n+1), n) = 1 \tag{4.13}$$

For a sufficiently large n (when the probability curve makes a sufficiently sharp transition), a TLN should be able to realize any dichotomy induced on $2(n+1)$ points. On the other hand it will most probably fail to realize any dichotomy on more than $2(n+1)$ points. It is this sudden transition that gives us a measure of the capacity of a TLN with $n+1$ weights, C_{max}:

$$C_{max} = 2(n+1) \tag{4.14}$$

for a TLN with n inputs or equivalently $(n + 1)$ weights including the threshold.

4.7 Revisiting the XOR Problem

In the light of the aforesaid discussion, we revisit the XOR problem and re-interpret our earlier solution put forth in Section 4.5.2. Recall that a threshold logic neuron is essentially a dichotomizer. The number of dichotomies $\mathcal{L}(p, n)$ of p points in n dimensions provides a direct measure of the number of Boolean functions defined on p points that are computable by a threshold logic neuron with n inputs. For example, a threshold logic neuron with 2 inputs (and a bias weight) can compute the entire set of Boolean functions that are defined on 3 points in $\mathbb{R}^2$. This is because the total number of functions that can be defined on 3 points is $2^3 = 8$, and the total number of dichotomies that can be generated by 3 points in 2 dimensions is $\mathcal{L}(3, 2) = 8$. Adding a fourth point increases the total number of possible functions to $2^4 = 16$, whereas the total number of dichotomies that can be generated with 4 points in 2 dimensions is only $\mathcal{L}(4, 2) = 14$. Two functions (for example, XOR and XNOR considering the functions defined on $\mathbb{B}^2$) are not computable by a TLN (because they are not linearly separable).

Then how can (or did) we solve the XOR problem? Our earlier design was logically implemented: Since (XOR = (OR) AND (NAND)), knowing that each sub-function (OR, AND, NAND) is linearly separable we could implement each function with an individual TLN, and by appropriately directing the signals of the OR neuron and the NAND neuron as inputs to the AND neuron the problem was solved.

Since a TLN can solve only linearly separable problems, non-separable functions must be first transformed into separable ones. There are two ways to do this: reduce the number of points, or increase the dimension. The latter argument finds its justification in Cover's Theorem on the separability of patterns that essentially states that a complicated pattern classification problem when transformed into a high dimensional space is more likely to be linearly separable than in the original lower dimensional space [113].

However, to gain further insight into what is really going on we re-direct our attention to $\mathcal{L}(p, n)$. Notice that if we have a large number of dichotomies, the *probability* that the TLN will be able to solve a classification problem increases. After all there is a larger chance that some dichotomy will be able to solve the problem. Now since a TLN can solve only a linearly separable problem, any design must be based on the notion that to eventually solve a linearly non-separable problem we have to make it linearly separable. How can this transformation be effected ? There are two ways:

- If somehow we can reduce the number of points p, then it might be possible that the mapping from the reduced set of points into the range space may become linearly separable.
- We might increase the dimension since the number of possible linear dichotomies then increases, and the probability that the linearly non-separable problem at hand becomes linearly separable, increases.

We consider each of these methods in sub-sections 4.7.1 and 4.7.2 respectively.

4.7.1 Solution 1: Reduce the Number of Points

We return first to the architecture suggested in Section 4.5.2 to solve the XOR problem in two dimensions, redrawn for convenience in Fig. 4.17.

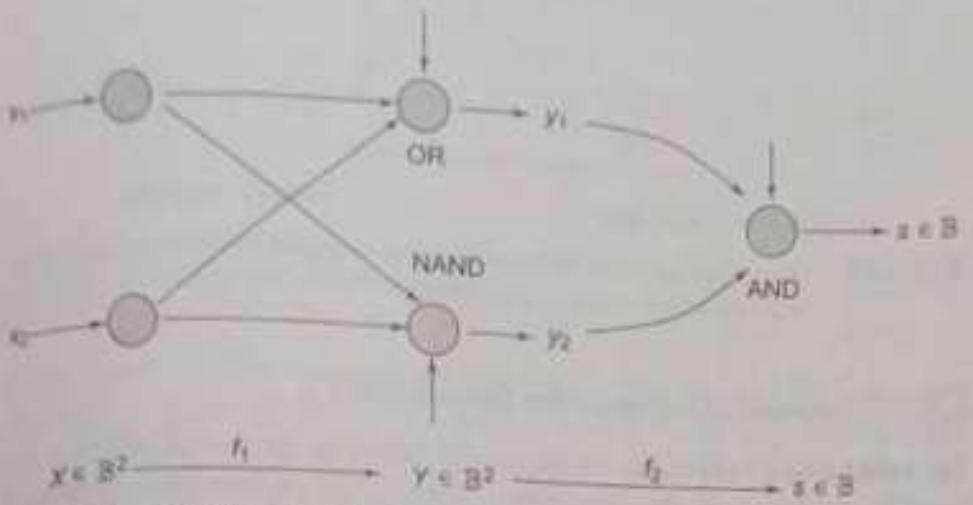


Fig. 4.17 An architecture that solves the XOR problem by effective reduction of the number of points

The logical functions implemented by each TLN (first layer nodes do not compute anything) is indicated in Fig. 4.17. Notice that the problem has been solved in two steps: first there is a map $f_1 : \mathbb{B}^2 \to \mathbb{B}^2$, following which there is a second map $f_2 : \mathbb{B}^2 \to \mathbb{B}$. Note that $Y = f_1(X)$ where $y_1 = x_1 + x_2$; $y_2 = \overline{x_1 x_2}$. This is shown in Table 4.2, and graphically in Fig. 4.18.

Table 4.2 Mapping 4 points to 3 points in two dimensions under f_1, makes the map f_2 linearly separable

x_1	x_2	y_1	y_2	z
0	0	0	1	0
0	1	1	1	1
1	0	1	1	1
1	1	1	0	0

Interestingly, the original four points $X = \{00, 01, 10, 11\} = \mathbb{B}^2$ are mapped under $f_1(\cdot)$ to the three points $Y = \{01, 10, 11\} = \mathbb{B}^2 \backslash \{0, 0\}$. Clearly, the final mapping from Y to s is linearly separable and is thus implementable by a single TLN. Reduction of one point in the same dimension has effectively converted a linearly non-separable problem into a separable one. This is precisely what layering achieves.

Here '\' is the set difference operation.

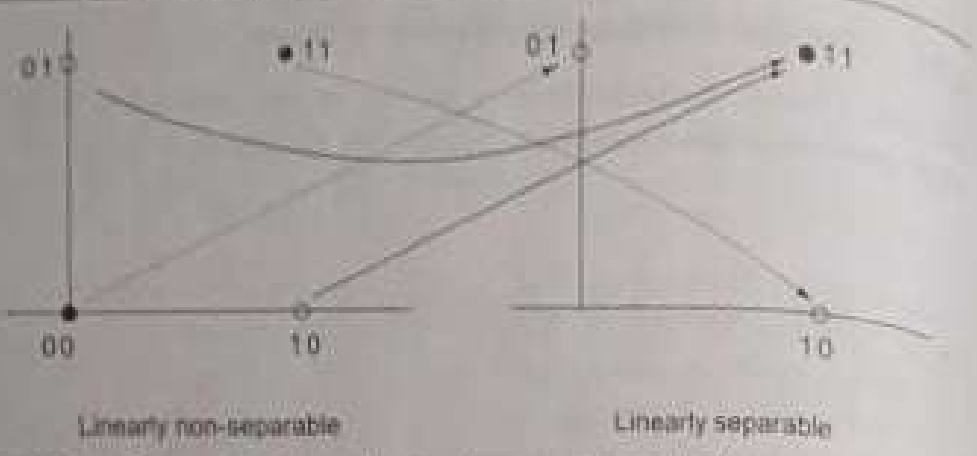


Fig. 4.18 Four points in two dimensions are mapped to three to make the XOR problem linearly separable

4.7.2 Solution 2: Increase the Dimension

The other way to solve the problem is to keep the number of points $p = 4$ fixed but to increase the dimension. This would increase the probability that the problem would become linearly separable. After all, $\mathcal{L}(4, 2) = 14$, but $\mathcal{L}(4, 3) = 16$. Thus there is a possibility that some dichotomy in the higher dimension might exist that solves this problem. And indeed it is so. This idea in fact motivates the following alternative architecture portrayed in Fig. 4.19.

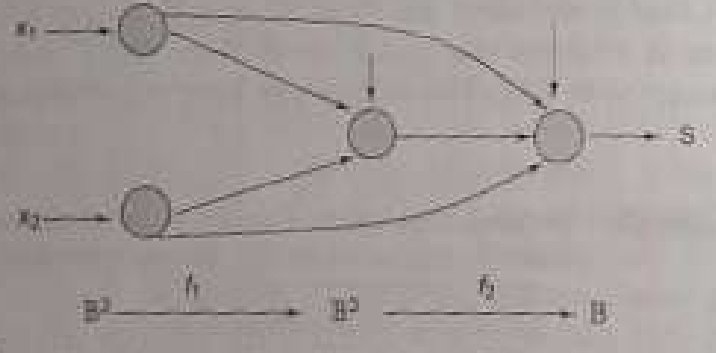
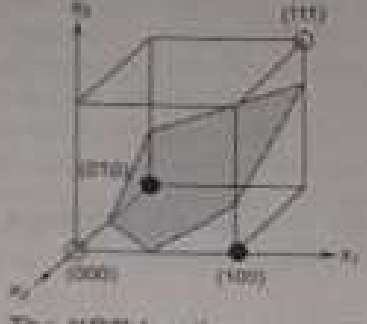


Fig. 4.19 An architecture that solves the XOR problem by effectively increasing the dimension of the pattern space



The XOR function mapped into three dimensions is linearly separable

We are now looking for an appropriate mapping $f_1 : \mathbb{B}^2 \rightarrow \mathbb{B}^1$ such that $f_2 : \mathbb{B}^1 \rightarrow \mathbb{B}$ is linearly separable. Finding this map is rather straightforward if we look carefully at the truth tables of the logical AND and XOR functions presented in Table 4.3.

We wish to add the appropriate third dimension x_3 which must be a function of x_1 and x_2. In fact setting $x_3 = x_1 x_2$ and adding this as the third dimension makes the problem linearly separable in three dimension as can be seen from the margin figure. Finding appropriate weights for the alternative architecture is now a straightforward task (see Review Question 4.7).

Table 4.3 The AND function added as the third dimension

x_1	x_2	$x_3 (= x_1 x_2)$	$x_1 \oplus x_2$
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

4.8 Multilayer Networks

As the above examples show, threshold logic neurons can be connected into multiple layers (or fields) in a feedforward fashion. This dramatically increases the computational capability of the system as a whole. We have already seen this in the case of the solution for the XOR problem.

In general, it is not uncommon to have more than one hidden layer when solving a classification problem. Such a situation arises when one has a number of disjoint regions in the pattern space that are associated into a single class. To understand this better, we make the following observations for binary threshold neurons with two hidden layers apart from the input and the output layer [356].

- Each neuron in the first hidden layer forms a hyperplane in the input pattern space.
- A neuron in the second hidden layer can form a hyper-region from the outputs of the first layer neurons by performing an AND operation on the hyperplanes. These neurons can thus approximate the boundaries between pattern classes.
- The output layer neurons can then combine disjoint pattern classes into decision regions made by the neurons in the second hidden layer by performing logical OR operations.

Figure 4.20 summarizes these observations graphically. In general, each increment of a neuron in any one of the hidden layers increases the number of connections in the system. This increases the number of degrees of freedom which will ultimately allow the machine to learn how to solve a complex decision making problem. It should be clear that no number of neurons in single hidden layer networks can suffice to separate meshed regions. However, two hidden layers suffice to form arbitrarily complex decision regions.

This observation is formally captured in the following theorem [355].

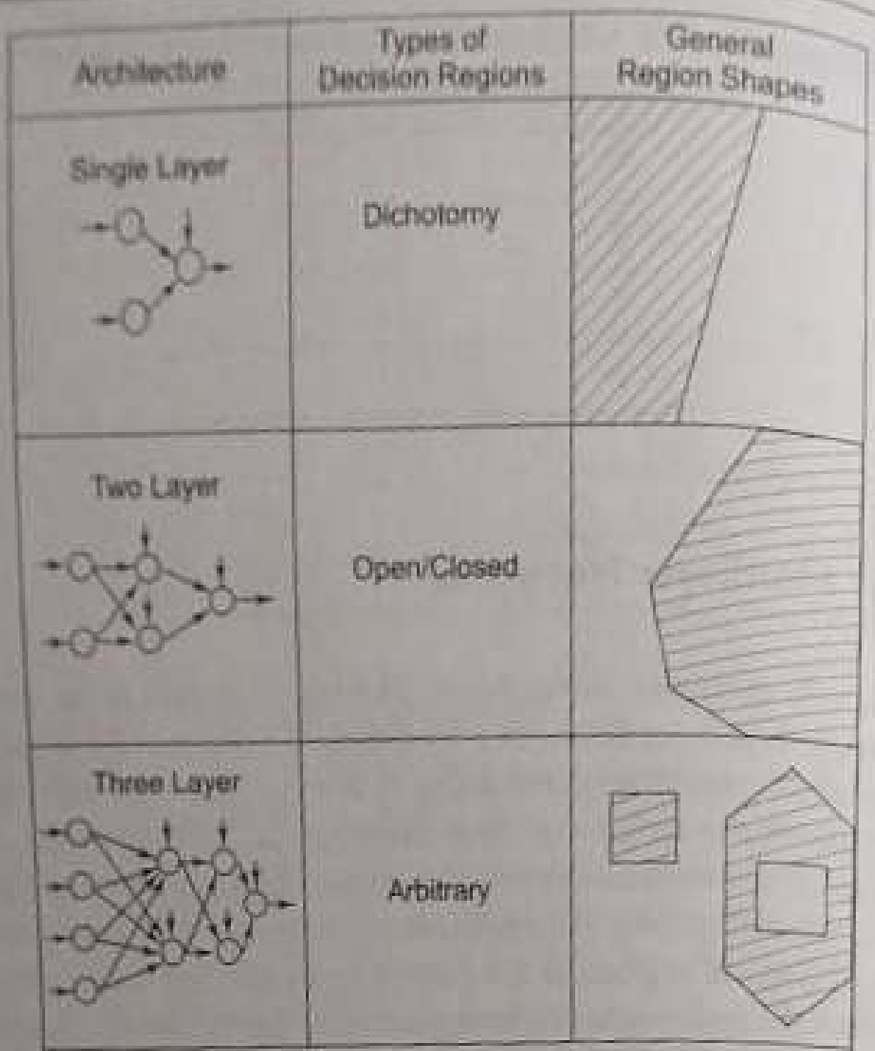


Fig. 4.20 The geometry of variously structured multilayered threshold neuron networks

Theorem 4.8.1

No more than three layers in binary threshold feedforward networks are required to form arbitrarily complex decision regions.

Proof: By construction. Consider the n-dimensional case: $X \in \mathbb{R}^n$. Partition the desired decision regions into small hypercubes. Each hypercube requires $2n$ neurons in the first layer (one for each side of the hypercube). One neuron in the second layer takes the logical AND of the outputs from the first layer neurons. Outputs of second layer neurons will be high only for points within the hypercube. Hypercubes are assigned to the proper decision regions by connecting the outputs of second layer neurons to third layer neurons corresponding to the decision region that the hypercubes represent by taking the logical OR of appropriate second layer outputs.

The role of hidden neurons is clearly very important: they generate hyperplanes that are the building blocks of decision regions. One might now ask the question: If we are given a specific number of hidden neurons in a single hidden layer network, then what is the maximum number of decision regions that the network can possibly partition the input space into? This represents some kind of a partitioning capacity of the network. In Section 4.9 we address this issue in some detail.

4.9 How Many Hidden Nodes are Enough ?

We continue to build upon our geometric point of view as regards the role of hidden neurons in creating appropriate partitions of pattern space. We restrict ourselves to a class of feedforward neural networks that employ binary threshold neurons arranged into a single hidden layer, since this is a well understood case. The role of hidden neurons when the signal functions are non-linear (sigmoids) is considerably more complicated, and we will return to this issue in Chapter 6.

What decision region geometry do hidden neurons carve out in pattern space? What is the maximum number of decision regions that can be separated by a single hidden layered network with h hidden nodes? To answer this last question, we closely follow the treatment in [75].

Consider a set of input patterns in n–dimensional Euclidean space $\mathbb{R}^n$. A hidden node in the first layer acts as an $(n-1)$ dimensional hyperplane that forms two decision regions. We assume that the n–dimensional space is linearly separable into M regions, if there exist M disjoint regions whose boundaries are composed of hyperplanes. M regions may be merged into C classes, $C < M$. Such a situation is portrayed in Fig. 4.21. As seen in Section 4.8, associating regions with classes is the task of the output nodes.

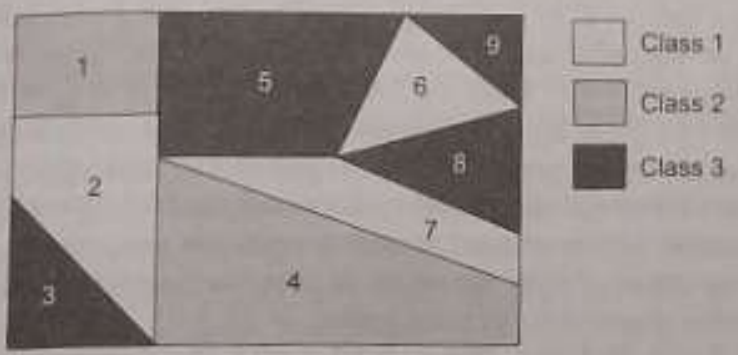


Fig. 4.21 Decision regions formed by a hyperplane geometry

We will now prove the following theorem by induction.

When we prove a proposition or hypothesis $P(n)$ by induction, we first prove it to be true for $n = 1$ using simple algebraic or geometric arguments. Having established this truth, we assume it to be true for some integer n, and then prove it to be true for $n+1$.

Theorem 4.9.1

(Cao and Mirchandani [75]). In n-dimensional space, the maximum number of regions that are linearly separable using h hidden nodes is

$$M(h,n) = \sum_{k=0}^{n} \binom{h}{k} \tag{4.15}$$

where

$$\binom{h}{k} = 0 \quad h < k \tag{4.16}$$

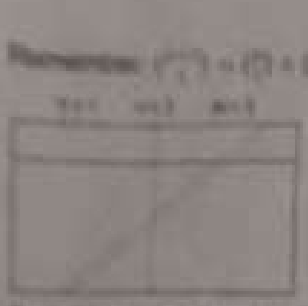
Proof By induction on n and on h.

- Consider the one dimensional case, $n = 1$. Here we show that $M(h, 1) = \binom{h}{0} + \binom{h}{1}$. Without any hidden node the entire space is treated as one region and therefore $M(0, 1) = 1$. Adding a single node gives $M(1, 1) = 2$ since a single hidden node dichotomizes the pattern space. Since each node adds a region in one dimension we have $M(p + 1, 1) = M(p, 1) + 1$. Using the induction hypothesis assume $M(p, 1) = \binom{p}{0} + \binom{p}{1}$. Then, since $M(p + 1, 1) = M(p, 1) + 1$, $M(p + 1, 1) = \binom{p}{0} + \binom{p}{1} + 1 = \binom{p}{0} + \binom{p}{1} + \binom{p}{0} = \binom{p+1}{0} + \binom{p+1}{1}$. Hence by induction, $M(h, 1) = \binom{h}{0} + \binom{h}{1} \; \forall \, h$.
- In two dimensions (see the figure alongside), a single hidden neuron dichotomizes $\mathbb{R}^2$. For this case the 1-dimensional expression, $M = \binom{h}{0} + \binom{h}{1} = 2$, applies. If $h = 2$, and if the separating lines are parallel, then $M = \binom{2}{0} + \binom{2}{1} = 3$. However, this expression holds good in two dimensions only if the lines are parallel. *Breaking the parallelism* generates a single extra region. Since each intersecting pair adds a new region, we have $M(2, 2) = M(2, 1) + \binom{2}{1} = 4$. Similarly if $h = 3$, the 1-dimensional result gives $M = \binom{3}{0} + \binom{3}{1} = 4$. However, three non-parallel intersecting lines can generate $M = 7$ regions. (Verify this yourself.) The number of extra regions generated by pairs of intersecting lines is now $\binom{3}{2} = \binom{3}{1} = 3$, whence $M(3, 2) = M(3, 1) + \binom{3}{2}$.
- In moving from two to three dimensions lines extend to planes which are orthogonal to the XY plane. For $h = 1$, $M = 2$ as in the 1 and 2-dimensional cases. For $h = 2$, $M = 4$ as in the 2-dimensional case. Notice that removing the orthogonality constraint does not add any extra regions in this case. Now consider $h = 3$. For the 2-dimensional case $M = \binom{3}{0} + \binom{3}{1} + \binom{3}{2} = M(3, 1) + \binom{3}{2} = M(3, 2)$. On breaking the parallelism (refer to the marginal figure) the central polyhedral cylinder generates an extra region, whereas the other six do not as they are half-open. Once again, $M(3, 3) = M(3, 2) + \binom{3}{3}$. Here, the number of extra regions generated is precisely the number of combinations of three planes out of three (each subset of three planes is capable of generating an extra region).
  For $h = 4$, $M(4, 3) = M(4, 2) + \binom{4}{3} = M(4, 2) + 4$. Hence by induction: $M(h, 3) = M(h, 2) + \binom{h}{3}$.
- Finally: since $P(1)$: $M(h, 1) = \binom{h}{0} + \binom{h}{1}$ has been proved for all positive integers h assume $P(n)$: $M(h, n) = \binom{h}{0} + \binom{h}{1} + \binom{h}{2} + \ldots + \binom{h}{n}$. Also, in moving from dimensions 1 to 2 and 2 to 3: $M(h, n + 1) = M(h, n) + \binom{h}{n+1}$. It follows therefore that $M(h, n + 1) = \binom{h}{0} + \binom{h}{1} + \ldots + \binom{h}{n} + \binom{h}{n+1}$. Since this holds good for $n+1$, it will hold for $n+2, \ldots$. Generalizing this yields:

$$M(h, n) = \sum_{k=0}^{n} \binom{h}{k}, \quad \binom{h}{k} = 0 \;\; h < k$$

Even the simplest of neurons have an interesting and powerful geometry. We have seen the limitations of simple binary neurons regarding the computation of elementary Boolean functions. Multilayered combinations

Remember: $\binom{h+1}{n+1} = \binom{h}{n} + \binom{h}{n+1}$



A single neuron represents a straight line in two dimensions, generating two decision regions.

With two nodes in two dimensions, breaking the parallelism adds an extra region. With $h = 2$, the number of extra regions is equivalent to the number of subsets of size 2 in a set of size 2; with $h - 1$ the number of extra regions is precisely the number of subsets of size 2 in a set of size 3.

These planes are orthogonal to the central plane

In three dimensions, lines extend to planes, which are orthogonal to the X-Y plane.

The central polyhedral cylinder generates an extra region upon removal of the orthogonality condition.

of such systems can compute powerful mappings by generating non-convex decision regions. Remember that the point of view projected throughout our discussion in this chapter is that of a neural network generating a map from an input space to an output space. An instance of weights of the entire network, defines the map. The geometric approach to the design of these weights was carefully considered in two and three dimensions. We also had to handle a very small number of domain points (4 or 8 at the most). But what of higher dimensions and a large number of patterns? What happens if you have to design a TLN to solve the XOR problem in five dimensions? This problem, motivates us to look for automated procedures to search for solutions to these high dimensional mappings. Chapter 5 will take us into the domain of algorithmic learning procedures for TLNs.

We now summarize what we have covered in this chapter.

Chapter Summary

- Pattern recognition is the discipline of research that attempts to build machines that search for structures in data sets in an attempt to provide a classification of the data.
- Linearly separable pattern sets are non-overlapping data points in the sense that their convex hulls are disjoint.
- The issue of linear separability is fundamental to the understanding of the classification capability of simple binary threshold neurons. Binary threshold neurons geometrize the input pattern space in the form of a dichotomy. This implies that they can classify only linearly separable pattern sets.
- Each neuron's activation equation essentially represents a hyperplane in pattern space which generates a dichotomy. The neuron weights can be designed using geometry by appropriately placing the hyperplane to separate the two classes in pattern space.
- Of the 16 Boolean functions in two dimensions, two are linearly non-separable: the XOR and the XNOR classification problems. To handle these linearly non-separable classification problems, more than one binary threshold neuron is required, and these need to be arranged into a layered architecture.
- The number of linear dichotomies that can be induced on p points in n–dimensions is $L(p, n) = 2\sum_{i=0}^{\min(n, p-1)} \binom{p-1}{i}$. This expression directly leads us to an estimate of the number of patterns that a TLN can store reliably. The capacity of a TLN is $C_{max} = 2(n + 1)$.
- The architectures to solve linearly non-separable problems are motivated by two ideas: decreasing the number of points; and increasing the dimension of the space.
- It can be proved that to solve any arbitrary classification problem, a three-layered network architecture suffices. (Given sufficient number of neurons!)

- In n-dimensional space, the maximum number of regions that are linearly separable using h hidden nodes (organized into a single hidden layer architecture) is $M(h, n) = \sum_{k=0}^{n} \binom{h}{k}$ where $\binom{h}{k} = 0, \quad h < k$. This gives us a measure of the partitioning capacity of the network.

Rosenblatt's famous Perceptron model [483] was one of the first neural network models that could learn elementary pattern classification tasks that involved linearly separable pattern sets. The computational limitations of Rosenblatt's Perceptron were later analyzed by Minsky and Papert in their book on *Perceptrons* [392].

The idea of the statistical pattern capacity of TLNs was first put forth by Cover in his 1964 Ph.D. thesis and a paper [113]. These ideas have been reviewed excellently by Nilsson in his book published in the same year [414]. The geometric ideas of multilayered network partitioning are summarized clearly in Lippmann's paper [355].

An excellent source on pattern classification techniques is Duda and Hart [136]. The book also discusses generalized linear discriminants nicely. For a recent overview of pattern recognition (PR) in the context of neural networks see Schalkoff [509] and Ripley [477]. A compendium on PR techniques related to fuzzy set theory is Bezdek's 1981 classic book [49], and Bezdek et al.'s recent book on fuzzy models and algorithms for PR [52].

Review Questions

4.1 Show that the weight vector corresponding to a separating hyperplane must be orthogonal to the hyperplane and pointing in a direction which indicates the hyperplane orientation (that is towards the region to be classified as a 1).

4.2 Apply the weight vector–hyperplane orthogonality idea to generate a solution to the NAND problem graphically.

4.3 Solve the logical OR problem geometrically to find appropriate weights and thresholds for the neuron.

4.4 How does the single neuron design for a NAND function differ from that of the AND neuron design? Correlate your answer to the weight vector–hyperplane orthogonality and orientation idea.

4.5 Solve the logical XNOR problem geometrically using a two layer TLN architecture to find appropriate weights and thresholds for different neurons.

4.6 Find a threshold logic neural network that can correctly solve the two class decision problem shown in Fig. 4.22.

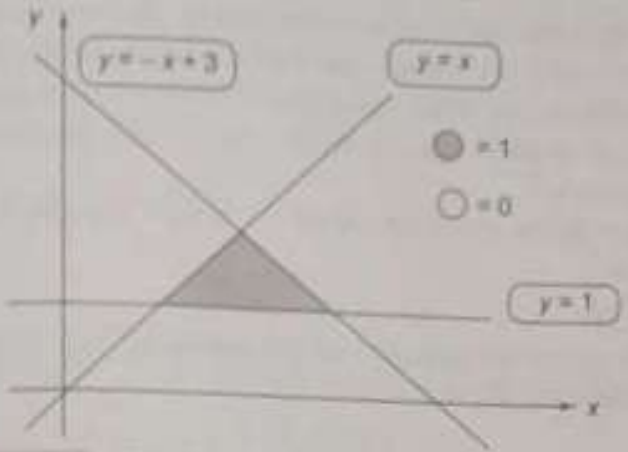


Fig. 4.22 Two class decision problem for Question 4.6

4.7 Solve the two dimensional XOR problem using the architecture shown in Fig. 4.19. This requires designing thresholds and input weight values for each of the neurons. Notice that skipping layers allows a certain degree of economy in terms of units.

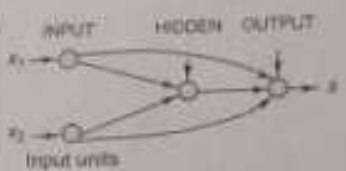


Figure 4.19 is repeated here for you: an alternative architecture to solve the two dimensional XOR problem.

4.8 Suggest a mapping other than the AND function that can be added as the third dimension to the 2-dimensional XOR problem in order to make it linearly separable. Based on this map design an appropriate network specifying the weights.

4.9 Using a network with 3 hidden units, solve the 3-bit even parity recognition problem: $\mathcal{P} : \mathbb{B}^3 \to \{0, 1\}$, such that $\mathcal{P}(B) = 1$ if B is of even parity, and $\mathcal{P}(B) = 0$ if B is of odd parity. (This is XNOR in three dimensions!)

4.10 Assume a binary threshold neuron with weights $w_1 = 1, w_2 = -1$ and $w_0 = 0$. See Fig. 4.23. Consider unit magnitude input vectors

MATLAB

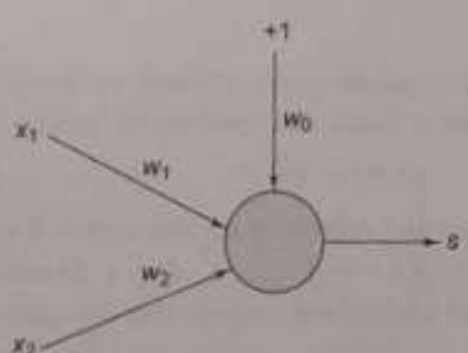


Fig. 4.23 Neuron architecture for Question 4.10

$X = (x_1, x_2)$ drawn from the circumference of the unit radius circle centred at the origin.

(a) Which inputs generate positive inner products and which ones generate negative inner products?

(b) Notice that the simple binary neuron "dichotomizes" the x_1–x_2 plane into two halves: one half corresponding to positive inner products; the other corresponding to negative inner products. What is the locus of input vectors that generate zero inner products?

(c) How do the results change if $w_0 = 0.5$? Use MATLAB graphics freely.

4.11 For a TLN:

(a) Show that the equation of the separating hyperplane Π can be written as (Eq. 4.17):

$$y(X) = X^T \hat{n} + \Delta_w = 0 \tag{4.17}$$

where Δ_w is the perpendicular distance of the hyperplane from the origin, $\hat{n}$ is the unit vector normal to Π, and X is any point on the hyperplane.

(b) What is the expression for the perpendicular distance from the origin to the hyperplane?

4.12 Give an example of a quadratic and generalized polynomial discriminant function.

4.13 How many hidden nodes would you select to solve the 3-class decision problem shown in Fig. 4.24? Justify your answer.

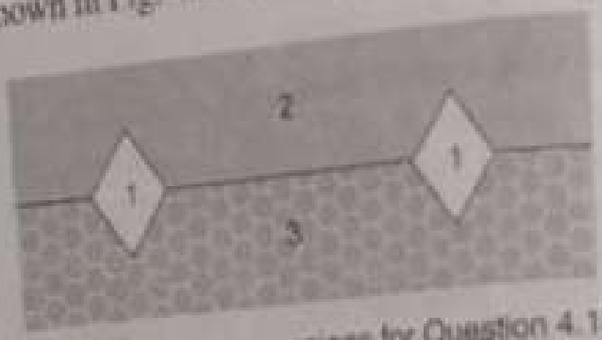


Fig. 4.24 Decision regions for Question 4.13

4.14 Consider a multiclass decision network as shown in Fig. 4.25. Each node implements a linear discriminant function:

$$y_i(X) = W_i^T X \qquad i = 1, \ldots, C \tag{4.18}$$

assuming augmented vectors. An input vector X is assigned to a class $\mathcal{C}_k$ if $y_k(X) > y_i(X), i = 1, \ldots, C; k \neq i$. Show that such a network will always lead to decision regions that are convex.

4.15 Based on the classification scheme of Question 4.14, draw the corresponding decision regions for the 3-class network of Fig. 4.26.

4.16 For the Cao–Mirchandani theorem derive the expression for $M(h, n)$ if

(a) $n = 1$

(b) $h \leq n$

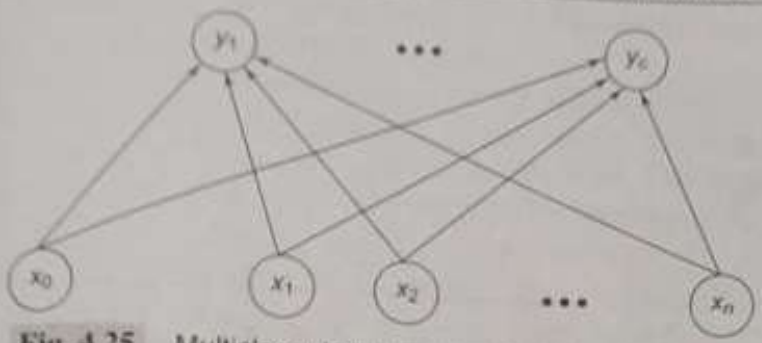


Fig. 4.25 Multiclass decision network for Question 4.14

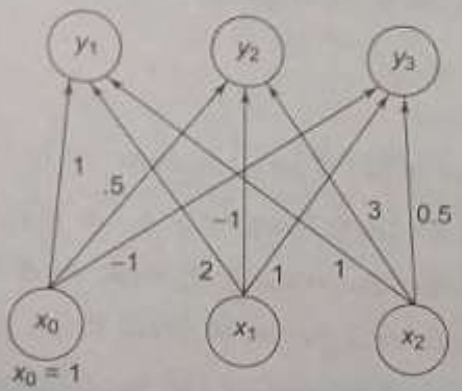


Fig. 4.26 3–class network for Question 4.15

4.17 Given h hidden nodes in 2 dimensions, employ the result of the Cao–Mirchandani theorem to provide a qualitative expression for the number of hidden nodes required to solve the XOR problem.